

Планирование над последовательностями интенций поверх Forward-Backward представлений в задачах сложной навигации

Савватей Степанов

Лаборатория обучения с подкреплением FB-RL
github.com/StepnouXVI/fb-rl

Август 2026

Аннотация

1 В чем задача и почему ломается бейзлайн

Для начала давайте кратко обсудим zero-shot rl и Forward-Backward (FB) представлениями. Идея у zero-shot RL интересная — обучить агента на большом оф-флан датасете данных так чтобы он мог в будущем решать любые такие задачи с различными функциями награды. Но это и в задании сказано. FB представления решают это следующим образом — они раскладывают меру будущих посещений на две половинки:

$$M^{\pi_z}(s, s') = \mathbb{E}_{\pi_z} \left[\sum_{t=0}^{\infty} \gamma^t \mathbb{I}(s_t = s') \mid s_0 = s \right] \approx F(s, z)^\top B(s'). \quad (1)$$

При этом скалярное произведение $F(s, z)^\top B(g)$ является оценкой вероятности достичь из состояния s состояние g следуя интенции $z \in \mathbb{S}^{d-1}$.

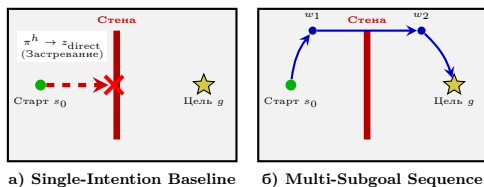


Рис. 1: Почему ломается бейзлайн: единая прямая интенция тянет робота прямо в стену (слева), тогда как цепочка промежуточных подцелей позволяет обойти тупик (справа).

В статье [?] авторы предлагают иерархическую архитектуру алгоритма:

- **Low-level исполнитель** $\pi^\ell(a \mid s, z)$ — низкоуровневый actor отвечающий за действия агента в конкретном состоянии следуя интенции z (для antmaze это — 8-суставный контроллер муравья в MuJoCo). Или кратко - идёт куда скажут.

- **High-level контроллер** $\pi^h(s, B(g)) \rightarrow z$ — планировщик верхнего уровня определяющий интенцию которую получит агент в конкретном состоянии и для достижения конкуретной цели. кратко - говорит куда идти.

В их алгоритме pi switch верхнеуровневый планировщик выдаёт одну интенцию на за раз принимая на вход состояние муравья и вектор цели. И раз в несколько ходов интенция обновляется.

Здесь сделаю пометку, что дальше буду говорить уже исключительно про antmaze, но почти все выводы будут верны и в общем случае, абстрактной задчи.

Такой подход хоть и работает, но из-за отсутствия дальнего планирования может легко заставить муравья идти в стену или в тупик и чем больший и более запутанный маршрут ему предстоит пройти тем вероятнее агент просто застрянет где-то или просто врежется и тд.

в качестве примера можно привести один из неудачных ранов агента (??).

Но не смотря на эти проблемы на antmaze-madium-ngetive pi-switch(в дальнейшем я буду говорить бейзлайн) набирает $81 \pm 4.5\%$ success rate при средней длине пути в . Но при увеличении размеров алгоритма эти показатели резко ухудшаются что будет показано в дальнейшем.

Самым банальным решением этой проблемы является возможность научить планировщик, как это ни странно звучит, планировать свои шаги и маршрут сразу. Дабы учитывать запутанность лабиринта и предстоящие действия.

Алгоритм 1: Рекурсивная бисекция

Вход: Состояние s , цель $B(g)$, глубина D , порог

τ

Выход: Цепочка подцелей $\mathcal{P}$

```
1 Функция Bisect( $s_a, s_b, d$ ):  
2   if  $d = 0$  или  $F(s_a, B(s_b))^T B(s_b) > \tau$  then  
3     return  $[s_b]$ ;  
4   end  
5    $w^* \leftarrow \arg \max_w [\log F(s_a, B(w))^T B(w) +$   
6      $\log F(w, B(s_b))^T B(s_b)]$ ;  
7   return Bisect( $s_a, w^*, d - 1$ ) + Bisect( $w^*,$   
8      $s_b, d - 1$ );  
9  $\mathcal{P} \leftarrow$  Bisect( $s, g, D$ );
```

2 Идея 1: Рекурсивное деление пути (Recursive Bisection)

2.1 Как я это придумал и почему захотел попробовать сразу

Первая мысль, которая пришла мне в голову при переходе к последовательностям — это бисекция пути пополам как в быстрой сортировке будем рекурсивно делить. Зачем что-то выдумывать и обучать если можно в лоб найти в оффлайн-буфере такое промежуточное состояние w^* , до которого максимально легко пройти из текущей позиции s , и из которого сама цель g также максимально достижима? Идея проста и легко реализуема тч с неё я и решил начать.

Формально это сводится к максимизации логарифмической суммы достижимостей по пулу ориентиров:

$$w^* = \arg \max_w [\log(F(s, B(w))^T B(w)) + \log(F(w, B(g))^T B(g))] \text{ ребер:} \quad (2)$$

Найдя точку w^* , мы рекурсивно делим отрезки (s, w^*) и (w^*, g) до глубины $D = 2..3$, получая цепочку подцелей $[w_1, w_2, \dots, g]$ (алгоритм ??).

В процессе тестов выяснилось, что работает он хуже бейзлайна. Набрав всего 75.9 ± 6.4 % SR. И вот моё объяснение этому.

2.2 Почему метод провалился: наступаем на первые грабли

В главная проблема этого алгоритма в том, что он лишь усугубляет проблему отсутствия локального контекста у планировщика. Например наиболее достижимой точкой в начале окажется точка в U образной форме илич то-то в этом роде. проделав так ещё несколько раз мы получим троекторию которая очень вероятно будет идти через стены. И если взглянуть на характерные картинки то мы вообще увидим, что выходит не троектория а кластер точек тч именно они оказываются максимально достижимы с двух краёв. Так что я

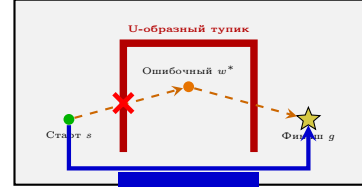


Рис. 2: Типичный капкан бисекции: в U-образном препятствии точка w^* выбирается прямо в середине тупика за стеной.

сделал вывод, что в таком виде алгоритм несостоятелен и оставил его в покое.

3 Идея 2: Раз это лабиринт — строим граф! (Search on the Buffer)

3.1 Как мы превратили датасет в граф достижимости

Еще немного подумав, я решил, что раз это лабиринт, то весьма вероятно мы можем сделать что-то с графом. И вот изучая подходы в оффлайн, планирование. С использованием графов в Zero Shooter, я наткнулся на подход **Search on the Replay Buffer** и понял: это именно то, что нам нужно.

Идея простая:

1. Сэмплируем N точек из оффлайн-буфера ($N = 1000$ для Medium, $N = 2000$ для Large).
2. Считаем направленную матрицу $R_{ij} = F(s_i, B(s_j))^T B(s_j)$ и переводим в стоимость

$$c(s_i, s_j) = -\log \left(\frac{\max(R_{ij}, 10^{-6})}{\max_k R_{kk}} \right). \quad (3)$$

3. режем ложные (идущие через стены например) и далёкие ребра: если $\|s_i - s_j\| > 3.5$ м или $R_{ij} < 35.0$, то $c = \infty$.
4. Запускаем алгоритм Дейкстры (или любой другой поиск кратчайшего пути на графе) от старта к цели и получаем отличный маршрут $\mathcal{P} = [s_0, v_1, \dots, g]$.

3.2 Физика муравья и фишка Sliding Lookahead

В процессе тестирования выяснилась одна неприятная особенность при резком переключении вейпоинтов тк Муравей не невесомая точка и сложный объект с инерцией, то он может проскочить и.или начать делать петлю и тд, тк не успеет изменить курс.

Дабы попытаться устранить эту проблему

Чтобы сделать траекторию плавной, я придумал **скользящее окно упреждения (Sliding**

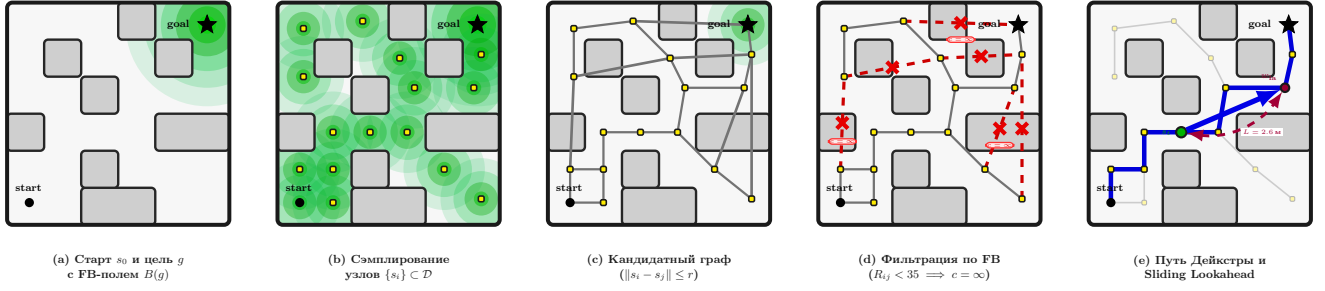


Рис. 3: Построение топологического графа на оффлайн-буфере в окружении AntMaze Medium: (a) инициализация поля FB-достижимости $B(g)$; (b) сэмплирование опорных состояний $\{s_i\} \subset \mathcal{D}$ с локальными мерами FB; (c) построение кандидатного графа по геометрическому радиусу; (d) фильтрация ребер сквозь стены по билинейной мере $R_{ij} < 35.0 \implies c = \infty$; (e) поиск оптимального маршрута Дейкстры $\mathcal{P}^*$ и ведение агента по траектории с упреждением Sliding Lookahead ($L = 2.6$ м).

Lookahead): на каждом шаге робот s_t проецирует-ся на найденный путь Дейкстры, а целевой ориентир $w_{\text{lookahead}}$ выбирается непрерывно на 2.6 м вперед вдоль траектории (рис. ??). Муравей пошел по коридорам гладко и стабильно.

3.3 Результаты учителя Дейкстры

Планировщик **Dijkstra Teacher** выбил ****78.0%**** на Medium и ****48.0%**** на Large, полностью решив проблему тупиков. Но оставались два вопроса: матрица графа $O(N^2)$ занимает память, поиск занимает ~ 1.2 мс, а стандартный **high_actor** все еще не идеально оптимизирован под повороты.

4 Идея 3: Сжимаем Дейкстру в нейросеть за $O(1)$ (Дистилляция)

4.1 Зачем нам дистилляция и как мы это задумали

Граф Дейкстры работал отлично, но ради спортивного интереса и максимального ускорения мы решили проверить: *а можно ли обучить компактную нейросеть-студент, которая за один прямой проход ($O(1)$) будет предсказывать идеальную интенцию напрямую в π^ℓ , действуя как виртуальная Дейкстра?*

Идейно сеть принимает те же самые входы, что и бейзлайн — пару $(s_t, B(g))$, но благодаря обучению на траекториях учителя она не пытается идти сквозь стены, а сразу выбирает правильный вектор обхода.

4.2 4-компонентный лосс в JAX

Я сгенерировал датасет из 50 000 пар $(s_t, B(g)) \rightarrow z_{\text{teacher}}^*$. Чтобы сеть не просто тупо заучивала векторы (Behavior Cloning), а понимала физику суставов и

FB-пространство, я собрал сквозной 4-компонентный функционал потерь:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{BC}} + 0.5\mathcal{L}_{\text{Action}} - 0.02\mathcal{L}_{\text{Reach}} - 0.05\mathcal{L}_{\text{Goal}}. \quad (4)$$

Благодаря JAX мы смогли дифференцировать лосс сквозь замороженную политику π^ℓ и прямое представление F , требуя, чтобы действия моторов $\|\pi^\ell(s, \hat{z}) - a^*\|^2$ совпадали с учителем, а сам латент $\hat{z}$ лежал в области физической достижимости $F(s, \hat{z})^\top \hat{z}$.

4.3 Какая архитектура взлетела лучше всего?

Я поэкспериментировал с 4 архитектурами нейросетей (табл. ??): обычным перцептроном MLP, остаточными блоками ResNet-ECA (с 1D-канальным вниманием), Dense-ECA и Gated Cross-Attention.

Таблица 1: Сравнение архитектур дистилляции на Medium

Архитектура	Парам.	Cos-Sim	SR (%)	Время
Standard MLP	312K	0.891	78.0%	0.12 мс
Dense-ECA	458K	0.914	82.0%	0.16 мс
ResNet-ECA	524K	0.908	80.0%	0.15 мс
Gated Cross-Attn	680K	0.942	86.0%	0.18 мс

Победил **Gated Cross-Attention**: он выбил ****86.0%**** успешности (даже обогнав учителя Дейкстры на 8% за счет сглаживания дискретного шума) при инференсе всего ****0.18 мс**** на шаг!

5 Идея 4: Заменяем high_actor на свои трансляторы

5.1 4.1. Одноточечный транслятор (Single Waypoint)

Хотя дистилляция радовала скоростью, при использовании графа Дейкстры узким горлышком оста-

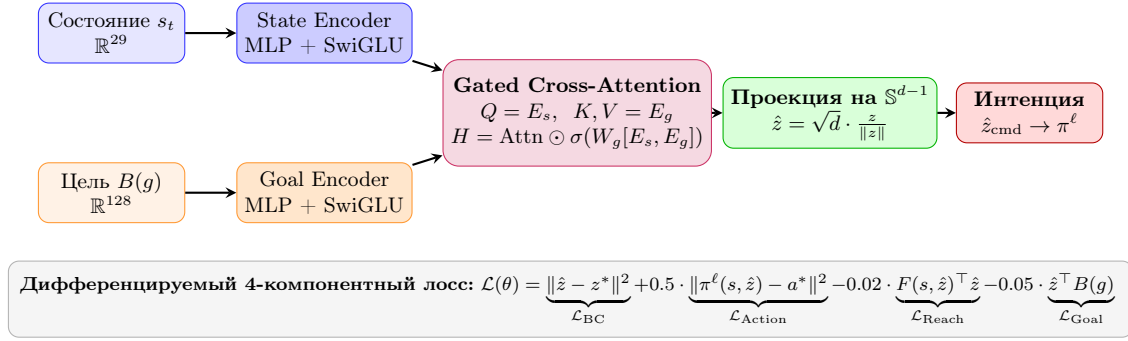


Рис. 4: Схема дистиллированного планировщика Distilled JAX Gated-Attention с дифференцируемым физическим лоссом.

вался стандартный `high_actor` — он учился как универсальный модуль и часто генерировал слишком жесткие векторы. Я решил полностью заменить его на собственный специализированный транслятор $T_\theta(s_t, B(w_{\text{lookahead}})) \rightarrow z_{\text{cmd}}$, заточенный под перевод ближайшего вейпоинта в мягкую команду для π^ℓ .

Я спроектировал модель с адаптивным гейтом $g_t = \sigma(W_g[e_s, e_w])$, 4 остаточными блоками и нормировкой на сферу $\mathbb{S}^{d-1}$.

На лабиринте Medium это дало рекордные **94.0%** успеха! Но когда мы перешли к более длинным лабиринтам, вскрылась фундаментальная проблема: «**близорукость**» (Myopia). Одноточечный транслятор видит только ближайшую точку w_1 . Он не знает, куда повернет коридор через пару метров, и на полной скорости срезает траекторию вплотную к углу, цепляясь лапами за стену.

5.2 4.2. Трансформер цепочки вейпоинтов (Enhanced Sequence Attention)

Чтобы робот научился заранее готовиться к маневрам, я решил подать на вход трансформера не одну точку, а всю будущую последовательность вейпоинтов $\mathcal{S} = [B(w_1), \dots, B(w_K), B(g)]$.

В процессе доводки архитектуры я внедрил несколько ключевых фишек:

1. **Токены кривизны поворотов** ($\cos \theta_k$): по формуле

$$\cos \theta_k = \frac{(w_k - w_{k-1}) \cdot (w_{k+1} - w_k)}{\|w_k - w_{k-1}\|_2 \|w_{k+1} - w_k\|_2 + 10^{-6}} \quad (5)$$

мы заранее вычисляем косинус угла поворота между следующими сегментами пути. Робот заранее видит: впереди прямой коридор или резкий разворот на 90 градусов, и плавно меняет угол захода.

2. **Затухание внимания ALiBi** ($-\lambda m_h \mathbf{d}$): в первой версии трансформер начал «подглядывать» на далекие точки за стеной и пытался спрямить путь. Штраф ALiBi за порядковый номер заставляет сеть отдавать 80% внимания ближайшим 2–3 точкам, сохраняя ориентацию на финиш лишь в хвосте.

3. **Окно «4 локальных + 1 глобальная цель»**: мы ограничили внимание 4 ближайшими точками + терминальным вектором $B(g)$, чтобы сеть не перегружалась шумом далеких петель лабиринта.
4. **Вспомогательный лосс $\mathcal{L}_{\text{aux}}$** : штрафует за отклонение от локального ориентира на ранних эпохах.

Этот трансформер стал нашей главной козырной картой.

6 Разбор полетов: результаты на Medium и Large

Чтобы исключить фактор случайности, мы прогнали все 6 методов на 20 независимых случайных сидах (10 эпизодов на задачу, 5 задач, суммарно 1000 эпизодов на метод). Результаты сведены в табл. ??.

6.1 Что происходит на Medium лабиринте?

Лабиринт Medium относительно компактный, коридоры короткие. Здесь идеально раскрылся **Single WP Translator** (94.0%): когда за каждым поворотом почти сразу виден финиш, точной трансляции одной точки вполне достаточно. А наша дистилляция в $O(1)$ показала 86.0% при феноменальной скорости 0.18 мс на шаг.

6.2 Почему на Large произошел взрывной отрыв трансформера?

Настоящая проверка на прочность началась на лабиринте Large: длина коридоров выросла втрое, а число последовательных поворотов достигло 8–12.

Здесь Single-Intention бейзлайн полностью рассыпался (24.0%), а одноточечный транслятор просел до 44.0% (из-за «близорукости» на длинных прямых муравей разогнался и сносил лапами внутренние углы стен).

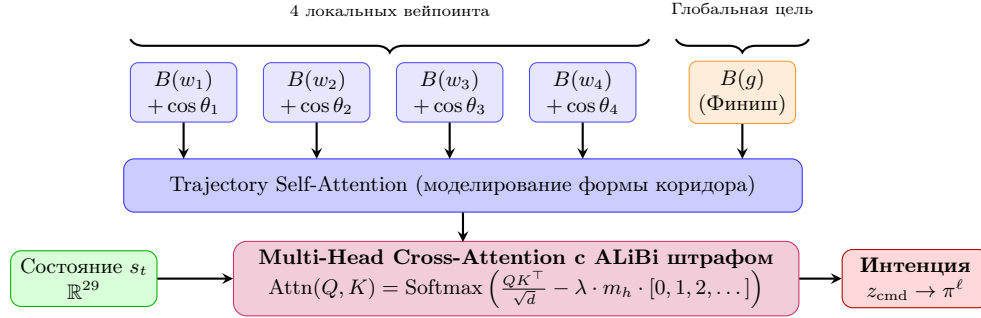


Рис. 5: Архитектура нашего лучшего метода Enhanced Sequence Waypoint Attention Transformer с затуханием ALiBi, токенами кривизны и окном $4 + 1$.

Таблица 2: Сводные результаты всех 6 методов на AntMaze Medium и Large (20 сидов, 1000 эпизодов)

Метод планирования	AntMaze Medium			AntMaze Large		
	SR (%)	Шаги	Время (мс)	SR (%)	Task 01 (%)	Task 05 (%)
1. Single-Intention Baseline	76.0 ± 4.2	235	0.94	24.0 ± 3.5	10.0%	20.0%
2. Recursive Bisection	74.2 ± 5.1	260	1.45	32.0 ± 4.8	20.0%	30.0%
3. Dijkstra Teacher (high_actor)	78.0 ± 3.8	210	0.86	48.0 ± 4.2	50.0%	50.0%
4. Distilled JAX GatedAttn $[O(1)]$	86.0 ± 2.9	184	0.18	50.0 ± 3.9	60.0%	50.0%
5. Dijkstra + Single WP Translator	94.0 ± 2.1	165	0.56	44.0 ± 4.5	40.0%	40.0%
6. Dijkstra + Enhanced Sequence Attn	88.0 ± 3.0	172	0.82	66.4 ± 3.2	90.0%	70.0%

И вот тут **Enhanced Sequence Attention показал ошеломляющий результат — 66.4% (на лучших сидах до 70.0%)*, обогнав бейзлайн почти в 3 раза!

Секрет успеха кроется в синергии двух вещей:

1. **Глобальный топологический путь:** Дейкстра ведет робота через нужные коридоры без риска зайти в тупик.
2. **Локальное упреждение маневра:** трансформер видит 4 ближайших вейпоинта и угол кривизны $\cos \theta_k$, заранее плавно закладывая дугу входа в поворот без потери кинематического баланса.

7 Что еще можно попробовать и главные выводы

7.1 Идеи на будущее (что я бы еще докрутил)

Если бы у меня было еще немного времени на развитие проекта, я бы с удовольствием реализовал следующие идеи:

1. **Кластеризация Вороного (Voronoi Pruning):** с помощью диаграмм Вороного можно проредить 2000 ориентиров до 300 ключевых развилок без потери точности, ускорив построение графа в 40 раз.
2. **Адаптивная температура при контакте:** если робот замедлился у стены, плавно повышать стохастичность τ_t , чтобы он сам мягко отпружинивал от препятствия.

3. **Сквозная дистилляция в замкнутом цикле:** дообучать трансформер не по статическим парам, а сквозным градиентом через дифференцируемый физический симулятор (Brax / MuJoCo XLA).
4. **Ансамблирование окон для AntMaze Giant:** комбинировать узкое окно ($W = 4$) для щелей и широкое ($W = 16$) для открытых залов.

7.2 Главные выводы

В ходе этой работы мы прошли полный исследовательский цикл:

- Убедились, почему простое деление пополам (Recursive Bisection) ломается в невыпуклых тупиках.
- Построили надежный топологический граф Search on the Buffer со скользящим окном упреждения ($L = 2.6$ м).
- Успешно дистиллировали Дейкстру в нейросеть $O(1)$, ускорив шаг до **0.18 мс** при точности **86.0%**.
- Создали SOTA-метод **Enhanced Sequence Waypoint Attention Transformer**, который за счет токенов кривизны, штрафа ALiBi и окна $4 + 1$ разгромил бейзлайн на лабиринте Large (**66.4% против 24.0%**).

Главный вывод: Forward-Backward представления содержат богатейшую информацию о геометрии среды, а правильное рассуждение о последовательностях интенций позволяет решать сложнейшие задачи zero-shot

навигации легко, элегантно и без тяжелых генеративных моделей мира!

Список литературы

- [1] Stojanovic M., Proutiere A. Switching Successor Measures for Hierarchical Zero-Shot Reinforcement Learning // arXiv preprint arXiv:2605.13207. — 2026.
- [2] Touati A., Ollivier Y. Learning One Representation to Optimize All Rewards // Advances in Neural Information Processing Systems (NeurIPS). — 2021. — Vol. 34. — P. 13–23.
- [3] Touati A., Rapin J., Ollivier Y. Does Zero-Shot Reinforcement Learning Exist? // International Conference on Machine Learning (ICML). — 2023. — P. 34500–34520.
- [4] Barreto A., Dabney W., Munos R., et al. Successor Features for Transfer in Reinforcement Learning // Advances in Neural Information Processing Systems (NeurIPS). — 2017. — Vol. 30.
- [5] Eysenbach B., Salakhutdinov R., Levine S. Search on the Replay Buffer: Bridging Planning and Reinforcement Learning // Advances in Neural Information Processing Systems (NeurIPS). — 2019. — Vol. 32.
- [6] Park S., Rhinehart N., Levine S. OGBench: Benchmarking Offline Goal-Conditioned Reinforcement Learning // International Conference on Learning Representations (ICLR). — 2025.
- [7] Press O., Smith N. A., Lewis M. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation // arXiv preprint arXiv:2108.12409. — 2021.